

ML System Design Document

UWB Ranging Error Correction Using DWM1000 Signal Diagnostics

Author: Austin Kilduff

Date: March 31, 2026

Course: Machine Learning - Week 10 Homework

This document describes the end-to-end ML system design for correcting systematic distance-measurement errors in a DWM1000 Ultra-Wideband (UWB) indoor positioning system. The system spans embedded firmware on Arduino Nano 33 BLE Sense Lite devices, a Raspberry Pi 5 BLE-to-web data pipeline, and scikit-learn ML models trained on rich signal-quality diagnostics to reduce ranging error from several-centimeter bias down to sub-centimeter corrected accuracy.

1. Problem Framing & Requirements

1.1 Problem Statement

Ultra-Wideband (UWB) time-of-flight ranging using the Decawave DWM1000 transceiver provides centimeter-level positioning in theory, but real-world deployments exhibit systematic distance errors of 1-3 meters caused by antenna delay miscalibration, multipath propagation, non-line-of-sight (NLOS) conditions, and environmental factors such as temperature drift. Our calibration datasets collected at a known 1.0 m reference distance with antenna delay 16700 show a mean measured distance of approximately 3.1 m (a +2.1 m bias), while at antenna delay 22750 the measurements swing to approximately -0.1 m (a -1.1 m bias). These errors render raw UWB readings unsuitable for precise indoor localization without correction.

1.2 ML Objective

The primary ML objective is regression: given a raw UWB distance measurement and its accompanying signal-quality diagnostics (received power, first-path power, signal quality index, CIR metrics, amplitude ratios, etc.), predict the true physical distance between the tag and anchor. A secondary objective is classification: predict the orientation angle of the tag relative to the anchor (0, 45, 90, 135, 180, 225, 270, 315 degrees) from the same signal features, since antenna orientation affects the channel impulse response.

1.3 System-Level Requirements

- Latency: Corrected distance must be available within 500 ms of each BLE packet arrival to support real-time dashboard visualization.
- Accuracy: Reduce mean absolute error (MAE) from the raw 1-2 m bias to under 10 cm after ML correction.
- Throughput: The system must handle 2 ranging packets per second per tag-anchor pair (one SS-TWR exchange every 500 ms).
- Portability: Models are serialized as Python pickle files and must load on a Raspberry Pi 5 with limited RAM (8 GB).
- Extensibility: Support multiple antenna delay configurations, multiple anchors, and future multi-tag deployments without retraining from scratch.

1.4 Stakeholders & Constraints

The system is designed for indoor positioning research. The primary constraint is that all ML inference runs on commodity hardware (Raspberry Pi 5 or a laptop) without GPU acceleration. The DWM1000 hardware is fixed and cannot be modified; all improvements must come from software-side calibration and ML correction. Data collection requires manual placement of the tag at known distances and angles, which limits the volume of labelled training data.

2. Data Pipeline Design

2.1 Data Source: Embedded Firmware

Data originates on Arduino Nano 33 BLE Sense Lite boards running custom firmware. The tag firmware (tag_T1.ino) performs Single-Sided Two-Way Ranging (SS-TWR) with each anchor, reads the DWM1000 diagnostic registers (RX power, first-path power, receive quality, standard noise, first-path amplitudes $fp_amp1/2/3$, CIR power, RX preamble accumulator count), and packs all fields into a 43-byte BLE characteristic (TagFrame struct). The anchor firmware (anchor_A1.ino / anchor_v4.ino) listens for poll messages, computes the reply delay, responds with a delayed transmission, and broadcasts its own 33-byte AnchorFrame with RX diagnostics over a separate BLE characteristic.

2.2 Data Ingestion: Raspberry Pi BLE Dashboard

A Python script (uwb_dashboard.py) runs on a Raspberry Pi 5 and acts as the central data ingestion gateway. It uses the Bleak library to continuously scan for BLE devices named T1-T10 (tags) and A1-A10 (anchors), connects to each, subscribes to BLE notifications, and deserializes the binary struct using Python's struct.unpack(). Each parsed packet is timestamped with ISO 8601 millisecond precision and stored in a thread-safe in-memory dictionary keyed by device name. A deque of the last 200 packets per device serves as the live history buffer. A Flask web server on port 5050 exposes GET /api/state returning the full device state as JSON, as well as POST /api/command for sending runtime commands (antenna delay, range interval) back to devices over BLE.

2.3 Raw Data Logging

Every received packet is logged to per-session CSV files in a logs/ directory organized by antenna delay setting (e.g., logs/AD_16700/, logs/AD_22750/). Tag packets produce tag_data_*.csv files with 20 columns (timestamp, device, anchor_id, seq, distance_m, rx_power, fp_power, fp_rx_ratio, quality, round_trip, reply_delay, std_noise, fp_amp1, fp_amp2, fp_amp3, cir_power, rxpacc, flags, anchor_count, nlos_suspect). Anchor packets produce parallel anchor_data_*.csv files with 16 columns. Log files are automatically rotated when the antenna delay changes or a new labelled collection session begins.

2.4 Labelled Data Collection

Two complementary tools produce labelled calibration datasets:

(a) Dashboard-side collection: The Flask API exposes POST /api/collection/start which accepts true_dist_m, angle_deg, notes, and target_samples. While active, every incoming tag packet is augmented with the true distance label and the computed error_m, then appended to a ble_cal_dataset.csv in the appropriate AD subfolder.

(b) Client-side collection: The uwb_ble_calibration.py Tkinter application polls the dashboard's /api/state endpoint (every 0.5 s), deduplicates packets by sequence number, and lets the operator start/stop labelled sessions. It can also bulk-import existing tag_data_*.csv log files and label them with a true distance and angle.

(c) Serial-based collection: The UWB_calibration_ml.py tool connects directly to the Arduino over USB serial, parses the 11-column CSV output, and stores labelled samples for scenarios where BLE is not used.

2.5 Feature Engineering

Beyond the 11 raw signal features available in each BLE packet, three engineered features are computed:

- $amp1_ratio = fp_amp1 / \text{mean}(fp_amp2, fp_amp3)$: first-path dominance indicator that helps distinguish direct-path from multipath-dominated channels.

- $\text{cir_norm} = \text{cir_power} / \text{rxpacc}$: normalized channel impulse response energy, removing the effect of varying preamble accumulation counts.

- $\text{ampl_spread} = |\text{fp_ampl2} - \text{fp_ampl3}|$: quantifies multipath spread by measuring the disparity between the two secondary amplitude peaks.

Additionally, the $\text{power_diff_dB} = \text{rx_power} - \text{fp_power}$ feature (fp_rx_ratio in BLE data) is used as a strong NLOS indicator. Non-finite values are imputed with column medians via a robust `safe_build_X()` function.

2.6 Data Storage & Versioning

Datasets are stored as flat CSV files organized by antenna delay configuration (`logs/AD_<value>/ble_cal_dataset.csv`). This partitioning ensures that models trained for one hardware calibration setting do not mix with data from another. The working dataset within the ML tool is kept in-memory as a pandas DataFrame and persisted to `ble_cal_dataset.csv` on each session save. Trained models are serialized using Python pickle (.pkl files), storing the full sklearn Pipeline, the feature list, and metadata (algorithm name, MAE, accuracy, etc.) in a single file.

3. Model Selection & Training

3.1 Algorithm Candidates

Four regression algorithms are offered in the training UI, chosen to span a range of model complexity:

1. Polynomial Regression (degree 1-6): Baseline linear model on polynomial-expanded features. Fast to train, fully interpretable, but cannot capture non-linear interactions between signal features.
2. Ridge Polynomial Regression (degree 3): Adds L2 regularization to polynomial regression, mitigating overfitting when the feature space expands combinatorially at higher degrees. Selected as a regularized baseline.
3. Gradient Boosting Regressor (GBR): The primary model. Configured with 400 estimators, max depth 4, learning rate 0.05, and 80% subsampling. GBR excels at capturing non-linear relationships and feature interactions without explicit polynomial expansion, and provides built-in feature importance rankings.
4. Random Forest Regressor: 400 trees, max depth 10. Included as an ensemble alternative to GBR with natural resistance to overfitting through bagging.

3.2 Justification of Primary Model (Gradient Boosting)

Gradient Boosting was selected as the default for several reasons grounded in course concepts from Weeks 1-9:

- Bias-variance tradeoff (Week 2): GBR sequentially reduces bias by fitting residuals of previous trees, while the low learning rate (0.05) and subsampling (0.8) control variance, directly applying the bias-variance decomposition principle.
- Regularization (Week 4): The `max_depth=4` constraint and `subsample=0.8` act as implicit regularization, preventing the model from memorizing noise in the signal diagnostics. This is analogous to L2 regularization in Ridge regression but applied structurally.
- Feature importance (Week 5): GBR natively provides Gini-based feature importance scores, enabling us to identify which DWM1000 diagnostics (e.g., `fp_rx_ratio`, `quality`, `ampl1_ratio`) contribute most to error correction, informing future hardware decisions.
- Cross-validation (Week 3): All models are evaluated with k-fold cross-validation ($k = \min(5, N/10)$) using negative MAE scoring, ensuring the reported performance is not inflated by a favorable train/test split.

3.3 Angle Classification

A secondary Random Forest Classifier (400 trees, max depth 12) is optionally trained to predict the tag's orientation angle from signal features. This classification task leverages the fact that the DWM1000 antenna radiation pattern is not omnidirectional: `fp_rx_ratio`, `ampl1_ratio`, and `ampl_spread` vary systematically with angle. The classifier uses an 80/20 train/test split and reports accuracy. This is relevant to Weeks 6-7 (classification metrics, confusion matrices). The angle classifier requires at least 2 distinct angle labels in the dataset to be activated.

3.4 Training Pipeline

Each model is wrapped in an sklearn Pipeline:

GBR pipeline: `StandardScaler -> GradientBoostingRegressor`

RF pipeline: StandardScaler -> RandomForestRegressor

Ridge pipeline: PolynomialFeatures(3) -> StandardScaler -> Ridge(alpha=1.0)

The Pipeline abstraction (Week 8) ensures that the scaler is fit only on training data and applied consistently at inference time, preventing data leakage. The train/test split is 80/20 with random_state=42 for reproducibility.

3.5 Evaluation Metrics

The distance corrector is evaluated using:

- Mean Absolute Error (MAE) in centimeters: primary metric, directly interpretable.
- Root Mean Squared Error (RMSE): penalizes large outlier errors more heavily.
- R-squared (R2): measures the proportion of variance explained by the model.
- Improvement percentage: $(\text{raw_MAE} - \text{corrected_MAE}) / \text{raw_MAE} * 100$.
- Per-distance breakdown: MAE computed separately for each true distance in the dataset, revealing whether the model generalizes across the measurement range.

The angle classifier is evaluated using accuracy (fraction of correctly predicted angles on the held-out test set).

3.6 Feature Selection

The default distance correction features are: distance_m (raw measurement), fp_rx_ratio, quality, ampl1_ratio, cir_norm, and rx_power. These were selected because they capture the primary error mechanisms: fp_rx_ratio is a strong NLOS indicator, quality and ampl1_ratio reflect multipath severity, and cir_norm normalizes for preamble length variation. The UI allows interactive feature selection so the operator can experiment with adding/removing features (e.g., std_noise, fp_ampl1, rxpacc) and observe the impact on MAE via the training report.

4. Deployment & Integration

4.1 System Architecture Overview

The deployed system consists of three tiers:

Tier 1 - Embedded (Arduino Nano 33 BLE Sense Lite):

Tag and anchor devices run C++ firmware performing SS-TWR and BLE telemetry. They communicate only via BLE advertisements and GATT characteristics. The tag firmware ranges at a configurable interval (default 500 ms) and broadcasts 43-byte TagFrames. The anchor firmware listens for polls and broadcasts 33-byte AnchorFrames. Both accept runtime commands over a BLE write characteristic (AD:<value> for antenna delay, RI:<value> for range interval, ST for status query).

Tier 2 - Edge Gateway (Raspberry Pi 5):

uw_b_dashboard.py runs as a persistent service, connecting to all BLE devices, logging raw data to CSV, and serving a real-time web dashboard (Flask, port 5050). The /api/state endpoint provides the live device state consumed by the ML tools. The /api/command endpoint enables remote reconfiguration of devices.

Tier 3 - ML Client (Laptop or Pi):

uw_ble_calibration.py and UWB_calibration_ml.py provide the calibration, training, and live inference UI. After training, the model is serialized as a .pkl file and can be loaded for real-time inference on incoming BLE packets.

4.2 Inference Flow

At inference time, the BLE poller in uw_ble_calibration.py receives new packets via the /api/state polling loop. For each packet:

1. Raw features are extracted (distance_m, rx_power, fp_power, etc.)
2. Engineered features are computed (ampl1_ratio, cir_norm, ampl_spread)
3. The feature vector is assembled and non-finite values imputed with column medians
4. The trained sklearn Pipeline (scaler + regressor) produces the corrected distance
5. Optionally, the angle classifier predicts the tag orientation
6. Results are displayed on the Live Inference tab: raw distance, corrected distance, correction error, estimated angle, and a 2D polar position map

The entire inference path (steps 1-5) executes in under 1 ms on a Raspberry Pi 5, well within the 500 ms latency budget between ranging events.

4.3 Model Serialization & Loading

Trained models are saved using Python's pickle module as .pkl files. The saved artifact contains the sklearn Pipeline (including fitted StandardScaler parameters and the trained regressor), the feature name list, and metadata (algorithm, MAE, accuracy). Loading a model restores all components; no retraining is required. The angle classifier (if trained) is saved alongside the distance corrector in the same pickle file.

4.4 BLE Command & Control Integration

The system supports closed-loop operation: the web dashboard and ML tools can send commands to embedded devices in real-time via BLE GATT writes. This enables:

- Remote antenna delay calibration (AD:<value>) without physically reprogramming
- Adjusting the ranging interval (RI:<value>) to balance throughput vs. power

- Querying device status (ST) to verify settings before data collection

Commands are queued in a thread-safe list, processed by the async BLE event loop, and responses are parsed to update the device settings state. The `set_all_delay` API endpoint broadcasts a delay value to all connected devices simultaneously.

4.5 Web Dashboard

The Flask-served HTML dashboard (`templates/dashboard.html`) polls `/api/state` every second and renders live device cards with connection status, packet counts, latest readings, signal quality gauges, and distance time-series charts. It also provides a command panel for sending AD/RI/ST commands and a data collection control interface that triggers the labelled collection API. The dashboard serves as the operator interface during both data collection campaigns and deployed inference monitoring.

5. Monitoring & Maintenance

5.1 Real-Time Signal Monitoring

The web dashboard continuously displays the following per-device metrics, enabling operators to detect anomalies before they corrupt data or inference:

- Connection status and uptime per device
- Packet count and packet rate (packets/second)
- Latest distance reading with NLOS suspect flag
- Signal quality metrics: rx_power, fp_power, fp_rx_ratio, quality index
- Distance time-series chart (last 200 packets) for visual drift detection

An NLOS flag is raised when $fp_rx_ratio < -6.0$ dB, indicating the first-path power is significantly below the total received power, a hallmark of severe multipath.

5.2 Data Quality Monitoring

Several mechanisms ensure data quality throughout the pipeline:

- The BLE poller deduplicates packets by sequence number, preventing double-counting.
- Non-finite values (inf, NaN) are replaced with NaN and imputed with column medians during feature extraction, as implemented in `safe_build_X()`.
- The collection UI shows live mean, standard deviation, and error statistics, allowing the operator to abort a session if signal conditions degrade.
- CSV log rotation by antenna delay ensures that data from different hardware configurations is never mixed in the same dataset file.
- The import tool validates that all required columns exist before accepting a CSV.

5.3 Model Performance Monitoring

During live inference, the system displays side-by-side raw and corrected distances with rolling error statistics. If the corrected error exceeds the raw error, it indicates model degradation (concept drift), alerting the operator to retrain. The training report includes per-distance MAE breakdowns and cross-validation scores with standard deviations, providing confidence intervals on model performance. Feature importance rankings highlight if a previously important feature (e.g., quality) loses relevance, suggesting hardware or environmental changes.

5.4 Auto-Reconnection & Resilience

The BLE event loop implements automatic reconnection with a 3-second retry interval. If a device disconnects (battery change, firmware reset, range loss), the handler re-scans for the device to handle BLE address changes, then reconnects and resumes notification subscriptions. The DWM1000 firmware includes a 5-second watchdog that triggers a soft reset (`dwmSoftReset()`) if no successful ranging exchange occurs, recovering from radio lockup conditions. The BLE dashboard runs periodic rescans every 15 seconds to discover devices that were powered on after startup.

5.5 Maintenance Procedures

- Retraining: When the environment changes significantly (new room, new furniture, temperature shift), collect a new labelled dataset at 2-3 known distances and retrain. The UI supports incremental dataset growth via import and append.
- Antenna delay recalibration: If raw measurements show consistent bias, use the dashboard's AD command to sweep antenna delay values and find the optimal setting before retraining.
- Log archival: Move old `AD_*` folders to archival storage to free SD card space.
- Firmware updates: Flash new .ino files via Arduino IDE; BLE reconnection ensures the dashboard picks up the device

automatically after reboot.

6. Challenges & Risk Mitigation

6.1 Data Collection Constraints

Challenge: Labelled data requires physically placing the tag at a known distance and angle, which is time-consuming and limits dataset size (typically hundreds, not thousands of samples).

Mitigation: The system provides three collection pathways (dashboard API, BLE client tool, serial tool) and a bulk import function to label existing CSV logs retroactively. Cross-validation with k-fold ensures reliable evaluation even with small datasets. The GBR model's subsampling (80%) and low learning rate further guard against overfitting on limited data.

6.2 Concept Drift & Environmental Change

Challenge: UWB signal characteristics change when the environment changes (moving furniture, temperature shifts, humidity, human presence). A model trained in one room may underperform in another.

Mitigation: The system is designed for rapid retraining. The complete collect-train-deploy cycle can be executed in under 10 minutes. The antenna delay partitioning in the data pipeline ensures that hardware configuration drift does not silently corrupt trained models. Live inference monitoring alerts the operator when corrected error exceeds expected bounds.

6.3 NLOS & Multipath

Challenge: Non-line-of-sight conditions cause severe ranging errors (sometimes 1+ meters) that are difficult to correct because the time-of-flight measurement itself is corrupted.

Mitigation: The firmware computes an NLOS flag ($fp_rx_ratio < -6$ dB) and includes it in every packet. The ML features `fp_rx_ratio`, `ampl1_ratio`, and `ampl_spread` are specifically designed to capture multipath severity, allowing the regression model to learn distance-dependent NLOS correction factors. The calibration data includes samples collected under realistic NLOS conditions to ensure the model has seen this distribution.

6.4 BLE Communication Reliability

Challenge: BLE connections are inherently unstable, especially in RF-noisy environments shared with UWB transmissions. Dropped connections interrupt data collection and inference.

Mitigation: The dashboard implements persistent auto-reconnection with BLE address re-scanning (handles Arduino reboots that may change the BLE address). The sequence number deduplication in the BLE poller ensures no data loss or duplication during brief disconnection/reconnection events. The command queue is processed asynchronously, preventing blocked I/O from stalling the main BLE event loop.

6.5 Model Generalization Across Hardware Configurations

Challenge: Different antenna delay settings produce fundamentally different error profiles (e.g., AD 16700 yields +2 m bias while AD 22750 yields -1 m bias). A model trained on one configuration fails on another.

Mitigation: Data is partitioned by antenna delay into separate directories (`logs/AD_16700/`, `logs/AD_22750/`). Models are trained per-configuration, and the antenna delay is stored as metadata in the calibration dataset. Future work could include antenna delay as a model feature to train a single generalizable model across configurations.

6.6 Pickle Serialization Security

Challenge: Python pickle files can execute arbitrary code when loaded, posing a security risk if model files are shared

between untrusted parties.

Mitigation: In the current single-user research deployment, pickle files are generated and consumed by the same operator on the same machine. For any future multi-user deployment, the serialization format should migrate to ONNX or sklearn's native joblib with hash verification.

6.7 Scalability

Challenge: The current system supports one tag-anchor pair. Scaling to multiple tags and anchors increases BLE connection management complexity and data volume.

Mitigation: The firmware and dashboard already support up to 10 tags (T1-T10) and 10 anchors (A1-A10) by design. The BLE event loop spawns independent async handlers per device, and the Flask API returns state for all devices simultaneously. The ML training pipeline operates on the consolidated dataset regardless of device count, as device identity is a metadata field, not a model input.

Tools & References: Arduino IDE, DW1000 Arduino library (Decawave/Thotro), Bleak BLE library, Flask, scikit-learn, pandas, NumPy, matplotlib. GitHub Copilot was used as a coding assistant during development.